

Il paradigma orientato agli oggetti

L'idea principale della OOP (Object Oriented Programming) risiede in buona parte nella rappresentazione della realtà, del mondo reale.

Facciamo un esempio per aiutarci a capire meglio: pensiamo ad una automobile.

Tutti noi sappiamo associare ad un'auto delle caratteristiche, ma molte poche persone (meccanici, costruttori, ingegneri, ...) sanno davvero come funziona ogni singola componente; quando decidiamo di andare a comprare un'auto nuova di certo non ci verrà in mente di conoscere il funzionamento accurato di ogni parte del motore, tantomeno andremo a chiedere al venditore di smontarlo per mostrarcelo.

Il concetto alla base della programmazione orientata agli oggetti è esattamente la stessa: non ci interessa sapere l'implementazione interna del codice, ma solamente sapere le **caratteristiche** e le **azioni** che determinate componenti sono in grado di offrire e svolgere. Questi componenti prendono il nome di **oggetti** e sono in grado di comunicare tra loro attraverso lo scambio di messaggi.

Ritorniamo al nostro esempio dell'auto, supponendo di voler realizzare un componente software in grado di riprodurla e gestirne le funzionalità. Possiamo definirne le caratteristiche: colore, marca, modello, motorizzazione, cilindrata, numero porte, chilometri percorsi, livello del carburante, posizione, marcia, ...

Per quel che riguarda le azioni, alcune possibili potrebbero essere: accelera, frena, sterza a sinistra, sterza a destra, cambia marcia, fai rifornimento, ...

Quindi è facile capire come le azioni siano semplicemente **tutte le cose che un oggetto è in grado di fare**, mentre le caratteristiche sono **tutti i dati che riguardano l'oggetto stesso** e che vengono utilizzati dall'oggetto stesso **per fare quello che ci si aspetta da lui**.

Nella programmazione ad oggetti le caratteristiche prendono il nome di **attributi**, mentre le azioni vengono dette **metodi**.

È necessario cambiare approccio rispetto alla programmazione procedurale: non avremo più un programma unico all'interno del quale inseriremo qualsiasi cosa, ma avremo degli oggetti, definiti separatamente, che interagiscono tra di loro richiamando i metodi di altri oggetti.

Inoltre, se con la programmazione procedurale siamo abituati a ragionare in maniera **top-down**, ovvero partire dal main e creare piano piano tutte le funzioni (procedure) che ci servono, con la programmazione orientata ad oggetti partiremo proprio dalla definizione delle classi e dalla creazione degli oggetti, e solo in un secondo momento (di solito all'ultimo) creeremo il programma principale, che si limiterà a creare oggetti delle diverse classi e a farli interagire tra di loro.

Questa è una metodologia che si applica bene nelle fasi di analisi e di progettazione di un problema: dal momento che il problema viene visto come un sistema, se ne costruisce un **modello**, con il quale si identificano gli elementi (oggetti) da cui è formato, con le loro caratteristiche ed i comportamenti che devono avere (non ci interessa ancora come i comportamenti verranno implementati).

Vantaggi della programmazione ad oggetti:

- il codice risulta essere più leggibile e comprensibile, anche per persone diverse dal programmatore che lo ha scritto;
- la manutenzione del programma è rapida (se questo è strutturato bene), ed è più semplice fare dei miglioramenti o delle correzioni;
- i programmi sono molto robusti, anche per la modellazione di sistemi complessi e con grandi quantità di dati;
- il codice è estremamente **riusabile** all'interno di altri programmi.

Attributi (o proprietà)

Abbiamo detto che gli attributi rappresentano tutte le caratteristiche e, in generale, i dati che sono associati ad un particolare oggetto e che gli servono per le sue elaborazioni.

Un errore comune che si può rischiare di commettere è associare ad un oggetto quante più proprietà possibili, convinti che questo possa aiutare a scrivere il programma; in realtà, tutto ciò è molto sbagliato, perché un oggetto deve comprendere solamente quelle proprietà che strettamente gli competono.

Pertanto, ogni volta che dobbiamo rappresentare/definire un oggetto ci dobbiamo porre la domanda "quali sono le caratteristiche, le proprietà, i dati che un oggetto necessita per riuscire a compiere le proprie azioni?".

Metodi

I metodi li abbiamo definiti come quell'insieme di azioni che il nostro oggetto è in grado di compiere.

Per capire questo la domanda che ci si pone è "quali azioni deve poter compiere il mio oggetto?".

Sono da evitare oggetti che non comprendano nessun metodo: è molto importante che ogni oggetto racchiuda non solo proprietà, ma anche metodi, che gli consentano di eseguire qualche azione. Di solito oggetti privi di metodi possono essere facilmente trasformati in attributi da associare ad altri oggetti.

Allo stesso modo bisogna cercare di non avere oggetti con troppi metodi, perché questo potrebbe essere un chiaro segnale che gli oggetti non sono "specializzati", ma compiono azioni che non competono loro.

Classi e oggetti

Uno dei concetti che stanno alla base della OOP è proprio quello di classe. Una classe può essere vista come una "entità" che raggruppa più oggetti che hanno le medesime proprietà e attributi.

Classe

La classe è la descrizione astratta degli oggetti attraverso attributi e metodi.

In un certo senso una classe è una **categoria di oggetti**, che in sostanza può essere vista come il tipo (di dato) degli oggetti che essa comprende.

Non bisogna, quindi, confondersi tra classi ed oggetti: un oggetto è una **istanza** di una certa classe.

Ogni istanza è **univoca**, ovvero ha una propria **identità**; questo significa che ogni oggetto è a sé stante, non dipende da altri oggetti. Inoltre, è possibile creare istanze separate di oggetti uguali, ovvero con lo stesso valore degli attributi, ma questo non obbliga ai due oggetti di doversi comportare allo stesso modo durante l'evoluzione del programma.

Nel nostro esempio, il concetto di automobile può essere facilmente associato ad una classe, mentre potrò creare delle istanze (degli oggetti), in cui il valore dei diversi attributi può essere diverso. Infatti, possiamo avere auto di marca diversa, con cilindrata diversa e che si alimentano in modo differente.

Naturalmente, gli attributi di una particolare istanza possono cambiare nel tempo (nel corso del programma). Se continuiamo a pensare all'auto, i chilometri percorsi varieranno in un qualche modo.

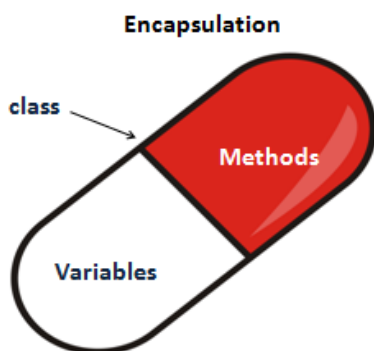
Possiamo quindi definire come **stato di un oggetto** l'insieme dei valori che assumono le sue proprietà in un certo istante di tempo. Anche una minima variazione comporta un cambiamento nello stato associato all'oggetto.

Invece, i metodi di un oggetto, che abbiamo detto rappresentare le azioni che quell'oggetto è in grado di compiere, vanno a definirne il **comportamento** (behavior).

Utilizzando le classi, tutto ciò che fa riferimento ad un oggetto è racchiuso all'interno della classe stessa, che diventa una unità di programmazione riutilizzabile più volte all'interno del programma o di altri programmi. Anche la manutenzione e l'aggiornamento risultano essere più efficaci, perché tutto ciò che serve è racchiuso all'interno della classe, sappiamo esattamente dove trovarlo. Tutto questo è definito dal concetto di incapsulamento.

Incapsulamento

Indica la proprietà degli oggetti di racchiudere tutte le informazioni ad essi relative: gli attributi, che ne definiscono i dati, e i metodi, che ne definiscono il comportamento.



In questo modo ogni oggetto risulta isolato dagli altri: si dice che attributi e metodi sono incapsulati in un oggetto.

Il principale vantaggio dell'incapsulamento consiste nel limitare gli effetti prodotti dalle modifiche di alcune parti del software: in sostanza, la modifica rimane isolata alla classe (e quindi agli oggetti di quella classe), senza che il cambiamento si ripercuota su altre classi o oggetti.

È estremamente importante che nessun oggetto vada ad accedere e manipolare direttamente i dati interni (le proprietà, gli attributi) di altri oggetti; per farlo, infatti, dovrà limitarsi a richiamare i metodi che quel particolare oggetto mette a disposizione (espone all'esterno).